NexusFin 数据库ER关系说明

一、总体概述

NexusFin 项目中，云卡和艾博生各自拥有一个独立的数据库，物理完全隔离，不存在跨库的物理外键约束。两个数据库通过**权益订单号（benefit_order_no）**作为逻辑纽带进行关联，实现业务数据的串联和追溯。

数据库	归属方	业务定位	表数量
`nexusfin_bridge`	云卡	中转撮合服务	6张
`nexusfin_equity`	艾博生	权益分发服务	7张

二、云卡数据库（nexusfin_bridge）

2.1 数据库职责

云卡作为三方业务的中转枢纽，其数据库承担的核心职责是：记录从艾博生流入的权益订单、向科技平台发起的试算和放款请求、撮合签约的合同存档，以及全链路的消息转发日志。所有数据围绕一张中转订单主表展开，形成以 order_transit 为核心的星型结构。

2.2 表结构与关键字段

（1）order_transit — 订单中转主表

这是云卡数据库的核心表，每一条记录代表一笔从艾博生流入、需要向科技平台发起借款的中转订单。它是云卡所有业务操作的起点和归宿，贯穿试算、签约、活体验证、放款的全流程。

关键字段	类型	说明
`order_no`	varchar, UK	云卡自身生成的中转订单号，全局唯一，是云卡内部所有表关联的主键纽带
`benefit_order_no`	varchar, FK	艾博生的权益订单号，**是跨库关联艾博生数据库的核心字段**
`external_user_id`	varchar	科技平台的用户ID，用于调用科技平台的试算、活体、放款等接口
`member_id`	varchar	艾博生的会员ID，用于与艾博生进行业务对账
	varchar	

`transit_status`		中转订单状态机，取值为 `INIT → TRIAL → SIGNED → GRANTING → GRANTED / FAILED`，是流程推进的核心驱动字段
`loan_order_no`	varchar	科技平台返回的借款订单号，放款成功后回填，用于后续还款状态的关联追溯

（2）trial_record — 试算记录表（待定）

记录云卡调用科技平台试算接口的每一次请求和响应。一笔中转订单可能因用户修改金额等原因进行多次试算，因此与 order_transit 是一对多关系。

关键字段	类型	说明
`trial_no`	varchar, UK	试算流水号，全局唯一，用于接口幂等和问题追溯
`order_no`	varchar, FK	关联中转订单号
`approved_amount`	decimal	科技平台批准的额度
`interest_rate`	decimal	年化利率
`term_count`	int	期数
`trial_status`	varchar	试算结果状态（`SUCCESS / FAIL`）
`trial_detail`	text	试算明细 JSON（含每期还款额等），用于 H5 页面展示

（3）sign_task — 撮合签约任务表

记录云卡调用电子签章服务创建的签约任务。一笔中转订单对应一个签约任务，是一对一关系（签约任务过期后可重新创建，但同一时刻只有一个有效任务）。

关键字段	类型	说明
`task_no`	varchar, UK	签约任务号，全局唯一
`order_no`	varchar, FK	关联中转订单号
`sign_url`	varchar	电子签章服务返回的签署页面 URL，用于引导用户在 H5 中完成签署
`sign_status`	varchar	签署状态（`PENDING → SIGNED / EXPIRED`）
`contract_type`	varchar	合同类型，固定为 `MATCH_AGREEMENT`（借款撮合协议）

（4）contract_info — 撮合协议合同存档表（可以去掉）

签约任务完成后，生成一份正式的合同存档。与 sign_task 是一对一关系。

关键字段	类型	说明
`contract_no`	varchar, UK	合同编号，全局唯一
`task_no`	varchar, FK	关联签约任务号
`file_url`	varchar	合同文件的 OSS 存储地址
`file_hash`	varchar	文件 SHA256 哈希值，用于防篡改校验

（5）liveness_proxy_log — 活体验证代理日志表（**本次可以忽略**）

记录云卡查询科技平台用户活体状态的每一次请求。一笔中转订单可能因活体过期需要多次查询，因此与 order_transit 是一对多关系。

关键字段	类型	说明
`request_no`	varchar, UK	请求流水号，全局唯一
`order_no`	varchar, FK	关联中转订单号
`liveness_status`	varchar	活体状态（`VALID / EXPIRED / PENDING`）
`need_reverify`	tinyint	是否需要重新验证（0否/1是），决定是否调起科技平台 App
`callback_status`	varchar	回调状态（`WAITING / RECEIVED`），标记是否已收到科技平台的活体验证回调

（6）notify_log — 消息转发通知日志表

记录云卡接收科技平台通知并转发给艾博生的每一条消息。一笔中转订单在放款和还款阶段会收到多条通知，因此与 order_transit 是一对多关系。

关键字段	类型	说明
`notify_no`	varchar, UK	通知流水号，全局唯一
`order_no`	varchar, FK	关联中转订单号
`notify_type`	varchar	通知类型（`GRANT_RESULT` 放款结果 / `REPAYMENT_STATUS` 还款状态）
`source`	varchar	消息来源（`CORE`，即科技平台）

`target`	varchar	转发目标（`EQUITY`，即艾博生）
`notify_status`	varchar	通知状态（`SUCCESS / FAIL / RETRYING`），支持重试机制
`retry_count`	int	已重试次数，配合最大重试策略使用

2.3 云卡内部关系总结

```
order_transit（中转主表）
  ├── 1:N ── trial_record（试算记录）
  ├── 1:1 ── sign_task（签约任务）
  │          └── 1:1 ── contract_info（合同存档）
  ├── 1:N ── liveness_proxy_log（活体代理日志）
  └── 1:N ── notify_log（消息转发日志）
```

order_transit 是整个云卡数据库的聚合根，所有子表都通过 order_no 与其关联。

这种设计确保了以"中转订单"为单位的数据完整性，便于按订单维度进行全链路查询和问题排查。

三、艾博生数据库（nexusfin_equity）

3.1 数据库职责

艾博生作为权益分发方，其数据库承担的核心职责是：管理静默注册的会员、记录权益订单的全生命周期（创建→签约→代扣→兜底→完成/退款）、存档电子签章合同，以及接收和处理来自云卡的放款/还款通知。数据围绕会员和权益订单两个核心实体展开。

3.2 表结构与关键字段

（1）member_info — 用户基础信息表

关键字段	类型	说明
`member_id`	varchar, UK	艾博生系统内部的会员ID，全局唯一，是艾博生内部所有业务表关联的主键纽带
`external_user_id`	varchar	科技平台的用户ID，用于跨系统的用户身份映射
`mobile_encrypted`	varchar	手机号（AES-256 加密存储）
`id_card_encrypted`	varchar	身份证号（AES-256 加密存储）
`real_name_encrypted`	varchar	真实姓名（AES-256 加密存储）

`member_status`	varchar	会员状态（`ACTIVE / INACTIVE`）
created_ts		
updated_ts		

> 接收到新用户（channel_code=KJ_PLATFORM_A, external_user_id=USR_A_12345, id_card=XXXX）
>
> Step 1: 用 id_card_hash 查询 member_info 表
>
> ├── 不存在 → 创建新会员（INSERT member_info），再创建渠道关联（INSERT member_channel）
>
> └── 已存在 → 查询 member_channel 表
>
> ├── 该渠道+外部ID已存在 → 幂等返回，直接返回 member_id
>
> └── 该渠道+外部ID不存在 → 只创建渠道关联（INSERT member_channel），关联到已有会员

（2）member_channel-会员主表

关键字段	类型	说明
`member_id`	varchar, UK	艾博生系统内部的会员ID，全局唯一，是艾博生内部所有业务表关联的主键纽带
`external_user_id`	varchar	科技平台的用户ID，用于跨系统的用户身份映射
`mobile_encrypted`	varchar	手机号（AES-256 加密存储）
`id_card_encrypted`	varchar	身份证号（AES-256 加密存储）
`real_name_encrypted`	varchar	真实姓名（AES-256 加密存储）
`member_status`	varchar	会员状态（`ACTIVE / INACTIVE`）
`created_ts`	DATETIME	
`updated_ts`	DATETIME	

（3）benefit_order — 权益订单主表

这是艾博生数据库的核心表，每一条记录代表一笔用户购买权益的订单。它承载了从订单创建、签约、首次代扣、兜底代扣到最终完成/退款的全生命周期状态。一个会员可以有多笔权益订单，与 member_info 是多对一关系。

关键字段	类型	说明
`member_id`		关联的会员ID
`channel_code`		渠道编码，如KJ_PLATFORM_A
`external_user_id`		外部平台的用户ID
`bind_status`		绑定状态：1正常 2解绑
`created_ts`		
`updated_ts`		

关键字段 (V7 新增/修改)	类型	说明
`agreement_no`	varchar	**新增**。权益协议号，在调用齐为【会员同步接口】时需要传入。
`qw_status`	varchar	**新增**

建议在代码中使用枚举类型来管理这些状态，并可以考虑在数据库中存储数字（如 10, 20, 21...）以提升查询效率，代码中做转换。

状态名	建议值	描述
`SYNC_PENDING`	10	会员同步处理中
`SYNC_SUCCESS`	11	会员同步成功
`SYNC_FAIL`	19	会员同步失败
`FIRST_DEDUCT_PENDING`	20	首次代扣处理中
`FIRST_DEDUCT_SUCCESS`	21	首次代扣成功
`FIRST_DEDUCT_FAIL`	29	首次代扣失败
`FALLBACK_DEDUCT_PENDING`	30	兜底代扣处理中
`FALLBACK_DEDUCT_SUCCESS`	31	兜底代扣成功

`FALLBACK_DEDUCT_FAIL`	39	兜底代扣失败
`EXERCISE_PENDING`	40	行权处理中
`EXERCISE_SUCCESS`	41	行权成功
`EXERCISE_FAIL`	49	行权失败
`REFUND_PENDING`	50	退款处理中
`REFUND_SUCCESS`	51	退款成功
`REFUND_FAIL`	59	退款失败
`REFUND_DONE`	60	无需退款（终态）

（4）benefit_product — 权益产品配置表

存储权益产品的基础配置信息，如费率、产品名称等。一个产品可对应多笔订单，与 benefit_order 是一对多关系。

关键字段	类型	说明
`product_code`	varchar, UK	产品编码，全局唯一
`product_name`	varchar	产品名称
`fee_rate`	decimal	费率（如 0.12 表示 12%）
`status`	varchar	产品状态（`ACTIVE / INACTIVE`），用于上下架控制

（5）payment_record — 支付与代扣流水表

记录艾博生通过宝付支付发起的每一笔代扣请求。一笔权益订单最多有两条支付记录（首次代扣 + 兜底代扣），因此与 benefit_order 是一对多关系。

关键字段	类型	说明
`payment_no`	varchar, UK	支付流水号，全局唯一，用于接口幂等和与宝付对账
`benefit_order_no`	varchar, FK	关联权益订单号
`payment_type`	varchar	支付类型（`FIRST_DEDUCT` 首次代扣 / `FALLBACK_DEDUCT` 兜底代扣），是区分两次代扣的核心字段
`amount`	decimal	代扣金额

`channel_trade_no`	varchar	宝付返回的渠道交易号，用于对账和退款
`payment_status`	varchar	支付状态（`PROCESSING → SUCCESS / FAIL`）
`fail_reason`	varchar	失败原因，首次代扣失败时记录，用于风控分析

变更：此表记录的不再是调用宝付的流水，而是调用齐为的支付流水。

关键字段 (V7 修改)	类型	说明
`channel_name`	varchar	**新增**。支付渠道名称，固定为 `QW` (齐为)。
`channel_trade_no`	varchar	**修改**。存储齐为返回的渠道交易号。

（6）sign_task — 签约任务表

记录艾博生调用电子签章服务创建的签约任务。一笔权益订单需要签署两份协议（《权益服务协议》和《权益先享后付协议》），因此与 benefit_order 是一对多关系。

关键字段	类型	说明
`task_no`	varchar, UK	签约任务号，全局唯一
`benefit_order_no`	varchar, FK	关联权益订单号
`sign_url`	varchar	签署页面 URL
`sign_status`	varchar	签署状态（`PENDING → SIGNED / EXPIRED`）
`contract_type`	varchar	合同类型（`EQUITY_AGREEMENT` 权益服务协议 / `DEFERRED_AGREEMENT` 先享后付协议）

（7）contract_info — 权益协议合同存档表

签约任务完成后，生成一份正式的合同存档。与 sign_task 是一对一关系。

关键字段	类型	说明
`contract_no`	varchar, UK	合同编号，全局唯一
`task_no`	varchar, FK	关联签约任务号
`contract_type`	varchar	合同类型（`EQUITY_AGREEMENT` / `DEFERRED_AGREEMENT`）

`file_url`	varchar	合同文件的 OSS 存储地址
`file_hash`	varchar	文件 SHA256 哈希值，用于防篡改校验

（8）notify_receive_log — 接收通知日志表

记录艾博生从云卡接收到的每一条通知消息（放款结果、还款状态等）。一笔权益订单在放款和还款阶段会接收多条通知，因此与 benefit_order 是一对多关系。

关键字段	类型	说明
`notify_no`	varchar, UK	通知流水号，全局唯一
`benefit_order_no`	varchar, FK	关联权益订单号
`notify_type`	varchar	通知类型（`GRANT_RESULT` 放款结果 / `REPAYMENT_STATUS` 还款状态）
`process_status`	varchar	处理状态（`RECEIVED → PROCESSED / FAILED`），标记艾博生是否已完成业务处理（如触发兜底代扣）

3.3 艾博生内部关系总结

```
member_info（会员表）
  └──── 1:N ── benefit_order（权益订单主表）
          ├──── N:1 ── benefit_product（权益产品配置）
          ├──── 1:N ── payment_record（支付流水）
          ├──── 1:N ── sign_task（签约任务）
          │          └──── 1:1 ── contract_info（合同存档）
          └──── 1:N ── notify_receive_log（接收通知日志）
```

艾博生数据库以 member_info 和 benefit_order 为双核心。member_info 是用户维度的聚合根，benefit_order 是订单维度的聚合根。所有子表都通过 member_id 或 benefit_order_no 与核心表关联。

四、跨库关系：云卡与艾博生的数据纽带

4.1 关联方式

云卡和艾博生的数据库之间不存在物理外键，两者通过**权益订单号（benefit_order_no）**进行逻辑关联。这是整个 NexusFin 项目中最核心的跨库关联字段。

```
nexusfin_equity（闻风）                          nexusfin_bridge（云卡）
┌──────────────────────────────────┐        ┌──────────────────────────────────┐
│                                  │        │                                  │
│  member_info ──1:N── benefit_order├──1:1──┤order_transit                     │
│                      │           │        │  │                               │
│              benefit_product     │        │  ├──1:N── trial_record           │
│                      │           │        │  ├──1:1── sign_task              │
│              ├──1:N─payment_record        │  │           └─1:1─ contract_info│
│              ├──1:N── sign_task   │        │  ├──1:N── liveness_proxy_log     │
│              │   └──1:1── contract_info    │  └──1:N── notify_log             │
│              └──1:N── notify_receive_log  │                                  │
│                                  │        │                                  │
└──────────────────────────────────┘        └──────────────────────────────────┘
                    │                                          │
                    └──────── benefit_order_no（逻辑纽带）──────────┘
```

艾博生是 benefit_order_no 的生成方，在用户完成权益签约后创建权益订单时生成。云卡是 benefit_order_no 的接收方，通过 API-04（推送权益订单）接口从艾博生获取，并存储在 order_transit.benefit_order_no 字段中。

4.2 关联关系

艾博生的 benefit_order 与云卡的 order_transit 是一对一关系：一笔权益订单对应且仅对应一笔中转订单。

4.3 数据流转路径

通过 benefit_order_no 这条逻辑纽带，可以实现从艾博生到云卡的全链路数据追溯：

艾博生侧追溯路径：

benefit_order_no → benefit_order → member_info（用户是谁）

benefit_order_no → payment_record（代扣了多少钱、是否成功）

benefit_order_no → sign_task → contract_info（签了什么合同）

benefit_order_no → notify_receive_log（收到了什么通知、是否处理）

云卡侧追溯路径：

benefit_order_no → order_transit → trial_record（试算了什么方案）

benefit_order_no → order_transit → sign_task → contract_info（签了什么撮合协议）

benefit_order_no → order_transit → liveness_proxy_log（活体验证是否通过）

benefit_order_no → order_transit → notify_log（转发了什么通知）

4.4 对账与一致性保障

由于两个数据库物理隔离，数据一致性通过以下机制保障：

实时层面，云卡在接收到艾博生推送的权益订单（API-04）后，立即创建 order_transit 记录，并同步返回确认。若创建失败，艾博生需要重试推送，接口通过 benefit_order_no 保证幂等。

日终层面，每日 T+1 对账任务会比对艾博生 benefit_order 和云卡 order_transit 中相同 benefit_order_no 的记录，校验订单状态、金额等关键字段是否一致。出现差异立即触发报警。

五、全局关系总览

```
nexusfin_equity（闻风）                          nexusfin_bridge（云卡）
┌──────────────────────────────────┐        ┌──────────────────────────────────┐
│                                  │        │                                  │
│  member_info ──1:N── benefit_order├──1:1──┤order_transit                     │
│                      │           │        │  │                               │
│              benefit_product     │        │  ├──1:N── trial_record           │
│                      │           │        │  ├──1:1── sign_task              │
│              ├──1:N─payment_record        │  │           └─1:1─ contract_info│
│              ├──1:N── sign_task   │        │  ├──1:N── liveness_proxy_log     │
│              │   └──1:1── contract_info    │  └──1:N── notify_log             │
│              └──1:N── notify_receive_log  │                                  │
│                                  │        │                                  │
└──────────────────────────────────┘        └──────────────────────────────────┘
                    │                                          │
                    └──────── benefit_order_no（逻辑纽带）──────────┘
```

整个数据模型的设计遵循三个核心原则：物理隔离（两个数据库完全独立部署，互不影响）、逻辑关联（通过 benefit_order_no 实现跨库追溯，不依赖物理外键）、聚合根清晰（云卡以 order_transit 为聚合根，艾博生以 benefit_order 为聚合根，所有子表围绕聚合根展开）。